

Resolución de la Actividad 10: Clase 05

Grupo 4

Integrantes:

- Avila, Aldana, legajo 1193721, alavila@uade.edu.ar
- Caivano, Alexander Francisco, legajo 1209389, acaivano@uade.edu.ar
- Casareski, Juan Ignacio, legajo 1176109, jcasareski@uade.edu.ar
- Segura, Juan Ignacio, legajo 1242159, jsegura@uade.edu.ar

Evento vs métrica

1. ¿Por qué no alcanza con guardar solo eventos?
El event log responde qué pasó con una instancia puntual. Las preguntas operativas (cuánto tarda en promedio un proceso, qué tenant genera más carga, cuándo hay picos) agregan miles de eventos por ventanas de tiempo. Calcular eso sobre Cassandra en cada consulta obliga a recorrer particiones enteras y reagregar en la aplicación. Una base de series temporales guarda la medición ya lista para agrupar por tiempo.
2. ¿Por qué no conviene usar InfluxDB como event log principal?
El event log es la fuente de verdad de la auditoría: necesita cada evento completo, con su payload, sin pérdida. InfluxDB está pensado para mediciones numéricas con retención limitada y downsampling; descartar detalle viejo es una virtud para métricas y un defecto grave para auditoría. Además la auditoría se consulta por instancia (la partición de Cassandra), no por ventana de tiempo.
3. ¿Qué métricas pueden derivarse de eventos?
Los conteos de instancias iniciadas y completadas, la duración de proceso (diferencia entre el evento de inicio y el de fin), la espera y el tiempo de resolución de tareas (diferencias entre eventos de tarea), los incumplimientos de SLA y los eventos por minuto.
4. ¿Qué métricas conviene registrar directamente?
Las que no existen como evento de negocio: la latencia de integraciones (la mide el servicio de integraciones al hacer cada llamada) y los usuarios concurrentes (se muestrean desde las sesiones en Redis).

Parte A: identificación de métricas

Métrica	Qué mide	Unidad	Fuente de datos	Uso operativo
instances_ started	Instancias iniciadas por intervalo	conteo	Evento de inicio de instancia	Detectar picos de carga y qué tenant los genera
instances_ completed	Instancias terminadas (aprobadas o rechazadas)	conteo	Evento de fin de instancia	Medir throughput; si crece menos que las iniciadas hay backlog
process_ duration	Tiempo total de una instancia completada	segundos	Diferencia entre inicio y fin al cerrar	Detectar procesos lentos y comparar versiones de la definición
task_wait_ time	Tiempo de una tarea en bandeja hasta que se empieza a resolver	segundos	Eventos de creación y toma de tarea	Detectar cuellos por rol, por ejemplo revisión legal
task_com- pletion_ time	Tiempo de resolución una vez tomada	segundos	Eventos de toma y cierre de tarea	Medir la carga real de trabajo de cada rol

Métrica	Qué mide	Unidad	Fuente de datos	Uso operativo
events_per_minute	Eventos de auditoría registrados por minuto	eventos/min	Consumidor del event log, pre-agregado	Medir la presión de escritura sobre Cassandra
integration_latency	Latencia de cada llamada a APIs externas	ms	Servicio de integraciones (api_afip, smtp_email)	Detectar degradación de AFIP antes de que frene validaciones
sla_violations	Tareas humanas que vencieron su SLA	conteo	Chequeo periódico de vencimientos	Alertar incumplimientos por proceso y rol
active_users	Usuarios con sesión activa	usuarios	Sesiones en Redis, muestreo por minuto	Dimensionar concurrencia (300 a 2.000 proyectados en la Actividad 4)

Preguntas guía:

1. ¿Qué métrica ayuda a detectar lentitud?
process_duration para el proceso completo; task_completion_time e integration_latency para localizar si la demora está en personas o en APIs externas.
2. ¿Qué métrica ayuda a detectar sobrecarga?
events_per_minute y active_users miden la presión sobre la plataforma; instances_started por hora muestra qué tenant la genera.
3. ¿Qué métrica ayuda a evaluar SLA?
sla_violations cuenta los incumplimientos; task_wait_time anticipa las tareas que están por vencer.
4. ¿Qué métrica sirve por tenant?
Todas llevan tenant_id como tag. La más directa es instances_started para comparar la carga relativa entre tenants.
5. ¿Qué métrica sirve por proceso?
process_duration agrupada por process_id, junto con sla_violations para ver qué procesos incumplen.
6. ¿Qué métrica sirve por rol?
task_wait_time y task_completion_time agrupadas por role muestran qué bandeja acumula espera.

Parte B: diseño de measurements

Measurement	Tags	Fields	Timestamp	Justificación
flowops_process_metrics	tenant_id, process_id, status	duration_seconds, steps_count	Cierre de la instancia	Duración y resultado por proceso y tenant, base del análisis de lentitud
flowops_task_metrics	tenant_id, process_id, task_type, role, status	wait_time_seconds, completion_time_seconds, sla_breached	Cierre o cambio de estado de la tarea	Espera y resolución por rol, y conteo de SLA vencidos con sum()

Measurement	Tags	Fields	Timestamp	Justificación
flowops_ event_rate	tenant_id	events_per_minute	Minuto medido	Presión de escritura sobre el event log, pre-agregada por el consumidor
flowops_ integration_ metrics	tenant_id, integration, status	latency_ms, retry_count	Momento de la llamada	Latencia y reintentos de api_afip y smtp_email, separadas por resultado
flowops_ usage	tenant_id	active_users, instances_started	Minuto muestreado	Gauges de carga por tenant para dimensionar concurrencia

Parte C: tags vs fields

Dato	Tag o field	Justificación
tenant_id	Tag	Casi toda consulta filtra o agrupa por cliente; cardinalidad acotada
process_id	Tag	Segunda dimensión de análisis; decenas de valores por tenant
task_id	Field	Identificador único de cardinalidad ilimitada; se guarda solo como referencia para cruzar con el event log
role	Tag	Pocos valores; agrupa las bandejas de trabajo
duration_seconds	Field	Valor medido sobre el que se calculan promedios y percentiles
status	Tag	Catálogo cerrado (completed, rejected, error); filtra sin inflar la cardinalidad
error_code	Tag	Catálogo acotado de códigos; agrupar por código detecta patrones de falla. Si fuera texto libre iría como field
latency_ms	Field	Valor medido
instance_id	Field	Igual que task_id: referencia para auditar, no dimensión de agrupación
sla_breached	Field	Valor 0 o 1; sum() cuenta violaciones por ventana sin abrir series nuevas

Preguntas guía:

1. ¿Qué datos se usan para filtrar o agrupar?
tenant_id, process_id, role, status y error_code: por eso son tags.
2. ¿Qué datos son valores medidos?
duration_seconds, latency_ms, los tiempos de espera y resolución y sla_breached: van como fields.

3. ¿Qué riesgo aparece si se usan tags con demasiada cardinalidad?
Cada combinación de valores de tags crea una serie con entrada propia en el índice. Millones de series degradan la memoria, la ingesta y las consultas.
4. ¿Conviene usar `instance_id` como tag?
No. Con la proyección de la Actividad 4 (300 tenants por 800 instancias diarias) se abrirían unas 240.000 series nuevas por día, casi todas de un solo punto. Va como field de referencia.
5. ¿Conviene usar `tenant_id` como tag?
Sí. Son entre 50 y 300 valores según la proyección, es la dimensión natural del análisis multi-tenant y no crece con el uso diario.

Parte D: cardinalidad

Dato	Cardinalidad esperada	¿Usarlo como tag?	Justificación
<code>tenant_id</code>	Media	Sí	50 a 300 valores proyectados; dimensión principal del análisis
<code>process_id</code>	Media	Sí	10 a 25 por tenant; combinado con <code>tenant_id</code> da como mucho unas 7.500 series, manejable
<code>instance_id</code>	Alta	No	Hasta 240.000 nuevas por día; series de un solo punto
<code>task_id</code>	Alta	No	Varias tareas por instancia; crece aun más rápido que <code>instance_id</code>
<code>role</code>	Baja	Sí	Un puñado de roles por tenant: solicitante, compras, legales, gerencia
<code>status</code>	Baja	Sí	Catálogo cerrado de estados
<code>user_id</code>	Alta	No	Miles de usuarios y crece con la adopción; el análisis operativo es por rol y el detalle por usuario queda en el event log

Preguntas:

1. ¿Qué es cardinalidad en este contexto?
La cantidad de series distintas de un measurement: cada combinación de valores de tags es una serie con índice propio.
2. ¿Por qué `tenant_id` puede ser un tag razonable?
Está acotado por el negocio (300 en la proyección a un año) y casi toda consulta filtra o agrupa por él.
3. ¿Por qué `instance_id` puede ser riesgoso como tag?
Su cardinalidad crece con el uso y sin techo. En un año el índice acumularía decenas de millones de series muertas que nadie vuelve a consultar.
4. ¿Por qué `task_id` puede ser riesgoso como tag?
Por lo mismo, multiplicado por la cantidad de tareas por instancia. Además ninguna consulta agrupa por una tarea puntual.
5. ¿Qué agregaciones podrían reducir cardinalidad?
Pre-agregar conteos por minuto (`events_per_minute`, `instances_started`), medir por rol y proceso en lugar de por tarea o usuario, y hacer downsampling con tareas de retención.

Parte E: ejemplos de line protocol

Líneas adaptadas al proceso alta de proveedor del tenant empresa_acme, con los roles e integraciones definidos en las actividades 2 y 7. En el protocolo cada punto ocupa una sola línea; acá se muestran partidas con sangría para que entren en la página. Timestamps en precisión de segundos (precision=s).

```
flowops_process_metrics,tenant_id=empresa_acme,process_id=alta_proveedor,  
  status=completed  
  duration_seconds=129600,steps_count=8 1781049600
```

```
flowops_task_metrics,tenant_id=empresa_acme,process_id=alta_proveedor,  
  task_type=human,role=compras,status=completed  
  wait_time_seconds=900,completion_time_seconds=1200,sla_breached=0 1781049660
```

```
flowops_task_metrics,tenant_id=empresa_acme,process_id=alta_proveedor,  
  task_type=human,role=legales,status=completed  
  wait_time_seconds=14400,completion_time_seconds=5400,sla_breached=1 1781049720
```

```
flowops_event_rate,tenant_id=empresa_acme  
  events_per_minute=320 1781049780
```

```
flowops_integration_metrics,tenant_id=empresa_acme,integration=api_afip,  
  status=error  
  latency_ms=4200,retry_count=2 1781049840
```

```
flowops_integration_metrics,tenant_id=empresa_acme,integration=smtp_email,  
  status=success  
  latency_ms=350,retry_count=0 1781049900
```

```
flowops_usage,tenant_id=empresa_acme  
  active_users=42,instances_started=12 1781049960
```

La tercera línea muestra el caso típico del TPO: la revisión legal esperó cuatro horas en bandeja y venció su SLA. La quinta registra una llamada a la API de AFIP que falló con reintentos, el dato que anticipa demoras en la validación de CUIT.

Parte F: consultas esperadas

Consulta de negocio	Measurement	Filtro / agrupación	Resultado esperado
Duración promedio de procesos por tenant	flowops_process_metrics	status=completed, agrupar por tenant_id, mean(duration_seconds) por día	Ranking de tenants con procesos más lentos
Cantidad de instancias iniciadas por hora	flowops_usage	sum(instances_started) en ventanas de 1 hora, por tenant_id	Curva de carga horaria y detección de picos
Tareas con mayor tiempo de espera por rol	flowops_task_metrics	Agrupar por role, mean y max de wait_time_seconds por día	La bandeja que acumula espera, hoy revisión legal
Eventos por minuto por tenant	flowops_event_rate	Agrupar por tenant_id	Qué tenant presiona más el event log
Latencia promedio de integraciones externas	flowops_integration_metrics	Agrupar por integration, mean(latency_ms) en ventanas de 5 minutos	Degradación de api_afip o smtp_email en cuanto empieza
Procesos con más incumplimientos de SLA	flowops_task_metrics	sum(sla_breached) por process_id y tenant_id, últimos 30 días	Ranking de procesos a rediseñar o re-dotar de gente

Ejemplo conceptual en Flux para la primera consulta acotada al caso del TPO: duración promedio por hora de alta_proveedor en empresa_acme, últimos 7 días.

```
from(bucket: "flowops")
  |> range(start: -7d)
  |> filter(fn: (r) => r._measurement == "flowops_process_metrics")
  |> filter(fn: (r) => r.tenant_id == "empresa_acme")
  |> filter(fn: (r) => r.process_id == "alta_proveedor" and r.status == "completed")
  |> filter(fn: (r) => r._field == "duration_seconds")
  |> aggregateWindow(every: 1h, fn: mean)
```

Parte G: retención y granularidad

Métrica	Granularidad inicial	Retención sugerida	Agregación posterior
Eventos por minuto	1 minuto	7 días en bruto	Promedio y máximo por hora, 90 días
Duración de tareas	Por tarea completada	30 días	Promedio y p95 por rol y día, 1 año
Duración de procesos	Por instancia completada	90 días	Promedio y p95 por proceso y día, 2 años
Latencia de integraciones	Por llamada	7 días	Promedio y tasa de error por hora, 90 días
SLA violados	Por evento	90 días	Total mensual por proceso y rol, 2 años
Usuarios concurrentes	1 minuto	14 días	Máximo por hora, 1 año

Preguntas:

1. ¿Qué métricas conviene conservar con detalle fino?
Las de diagnóstico reciente: latencia de integraciones y duración de tareas de las últimas semanas, donde importa el punto individual.
2. ¿Qué métricas pueden agregarse por hora o por día?
Eventos por minuto, usuarios concurrentes y las duraciones una vez pasada la ventana de diagnóstico: las preguntas sobre el pasado son de tendencia.
3. ¿Qué información perdería valor después de cierto tiempo?
El punto individual: la latencia de una llamada concreta de hace tres meses no cambia ninguna decisión. Si hiciera falta el detalle, está en el event log de Cassandra.
4. ¿Qué datos deberían mantenerse más tiempo por motivos de gestión?
Los SLA violados y la duración de procesos agregada: sostienen compromisos con clientes y comparaciones entre versiones de un proceso.
5. ¿Por qué la retención no debería ser igual para todo?
El costo de almacenamiento e índice crece con el detalle, y el valor del dato fino decae con el tiempo. Retener todo en bruto paga costo máximo por datos que solo se consultan agregados.

Parte H: integración con FlowOps

Subsistema FlowOps	Tecnología / modelo	Rol
Definiciones de procesos	MongoDB	Documentos versionados y flexibles por tenant
Estado de instancias y tareas	MongoDB, con Redis como caché	Fuente de verdad del estado actual
Eventos de auditoría	Cassandra	Historial inmutable append-only
Caminos y dependencias del flujo	Neo4j	Grafo de pasos, transiciones e integraciones
Caché, sesiones y colas	Redis	Baja latencia y TTL
Métricas temporales	InfluxDB	Series temporales, SLA y dashboards
Objetos con comportamiento	IRIS	Analizado en la Actividad 9; queda fuera del prototipo

Preguntas:

1. ¿Desde qué componente se generarían las métricas?
Desde el consumidor que ya procesa los eventos del motor para escribirlos en Cassandra: el mismo worker deriva las métricas y las manda a InfluxDB. El servicio de integraciones registra `integration_latency` al hacer cada llamada, y un job por minuto muestrea las sesiones activas desde Redis.
2. ¿Las métricas se escriben en línea o de forma asincrónica?
Asincrónica. El motor publica el evento y sigue; el escritor consume, arma las líneas y las envía en lotes. Una métrica con segundos de retraso no pierde valor.
3. ¿Qué pasa si InfluxDB no está disponible?
El escritor reintenta y, si la caída se extiende, encola con tope en Redis o descarta. El proceso de negocio no se entera. Las métricas perdidas se reconstruyen después desde el event log.
4. ¿Debe bloquearse la ejecución del proceso si falla el registro de métricas?
No. La observabilidad no participa del camino crítico: una aprobación no puede fallar porque el dashboard se quedó sin datos.
5. ¿Qué diferencia hay entre perder una métrica y perder un evento de auditoría?
La métrica es un dato derivado y reconstruible; perderla degrada la visibilidad de una ventana de tiempo. El evento es la fuente de verdad de la auditoría: perderlo deja un agujero en la trazabilidad que ninguna otra base puede reponer.

Parte I: decisión arquitectónica

Decidimos implementar InfluxDB en el prototipo, con un alcance acotado. Las preguntas del contexto (carga por tenant, duración de procesos, SLA incumplidos) son las que un cliente le haría a FlowOps en una demo, y sin una base de series temporales el prototipo no las responde: derivarlas de Cassandra en el momento exige recorrer particiones enteras. El costo marginal es bajo porque el equipo ya operó un stack equivalente de observabilidad (Grafana y Tempo provisionados por docker compose en otro proyecto) y el patrón se replica igual con InfluxDB como datasource.

El alcance en el prototipo se limita a dos measurements, `flowops_process_metrics` y `flowops_task_metrics`, escritos por lote desde el consumidor de eventos, y un dashboard de Grafana con paneles de duración por proceso, espera por rol e incumplimientos de SLA. `flowops_integration_metrics`, `flowops_event_rate` y `flowops_usage` quedan diseñados pero fuera del alcance.

Opción elegida	Justificación	Riesgo	Próximo paso
Implementado en el prototipo	Las métricas operativas son parte del valor del producto y el costo es bajo: dos servicios en el compose con un patrón ya probado por el equipo, y escritura asíncrona que no toca el camino crítico	Quinta tecnología del stack: más superficie para instalar y defender. Se mitiga acotando a dos measurements y sin código de consulta propio, Grafana consulta directo	Sumar InfluxDB y Grafana al docker-compose del prototipo, provisionar el datasource y escribir el consumidor de eventos a line protocol

Servicios a agregar al compose del prototipo:

```

services:
  influxdb:
    image: influxdb:2.7
    ports: ["8086:8086"]
    environment:
      - DOCKER_INFLUXDB_INIT_MODE=setup
      - DOCKER_INFLUXDB_INIT_ORG=flowops
      - DOCKER_INFLUXDB_INIT_BUCKET=metrics
    volumes:
      - influxdb-data:/var/lib/influxdb2

  grafana:
    image: grafana/grafana:latest
    ports: ["3000:3000"]
    environment:
      - GF_AUTH_ANONYMOUS_ENABLED=true
      - GF_AUTH_ANONYMOUS_ORG_ROLE=Admin
    volumes:
      - ./grafana-datasources.yml:/etc/grafana/provisioning/datasources/datasources.yml
    depends_on:
      - influxdb

volumes:
  influxdb-data:

```